1. **Цель работы**

   Получение умений и навыков проектирования объектнореляционных баз с учётом объектных расширений на примере PostgreSQL.

2. **Задание**

   Спроектировать физическую модель базы данных, находящуюся в третьей нормальной форме и включающей наследование и, хотя бы один пользовательский тип в соответствии с заданным вариантом. Написать соответствующий скрипт создания базы данных.

   Согласно варианту №6:

   поликлиника с учетом различного медицинского персонала):прием, пациент, процедура/прием, врач, стоимость, аппаратура. На приеме может быть только врач а процедуры могут вести и медсестры

   а. пациенты, приходившие на любые процедуры, , аппаратура для которой начинается со слова «лазер» , но это не единственная часть слова

   б. кабинет, где в один день проходят приемы нескольких врачей у одного пациента

   в. аппараты процедур с наибольшей стоимостью

   г. сотрудник поликлиники, работавший с наибольшим количеством различных пациентов в текущем году.

   д. пациент, не ходивший на процедуры к медсестре Ивановой

3. **Физическая модель**

   Разработанная физическая модель показана на рисунке 1.

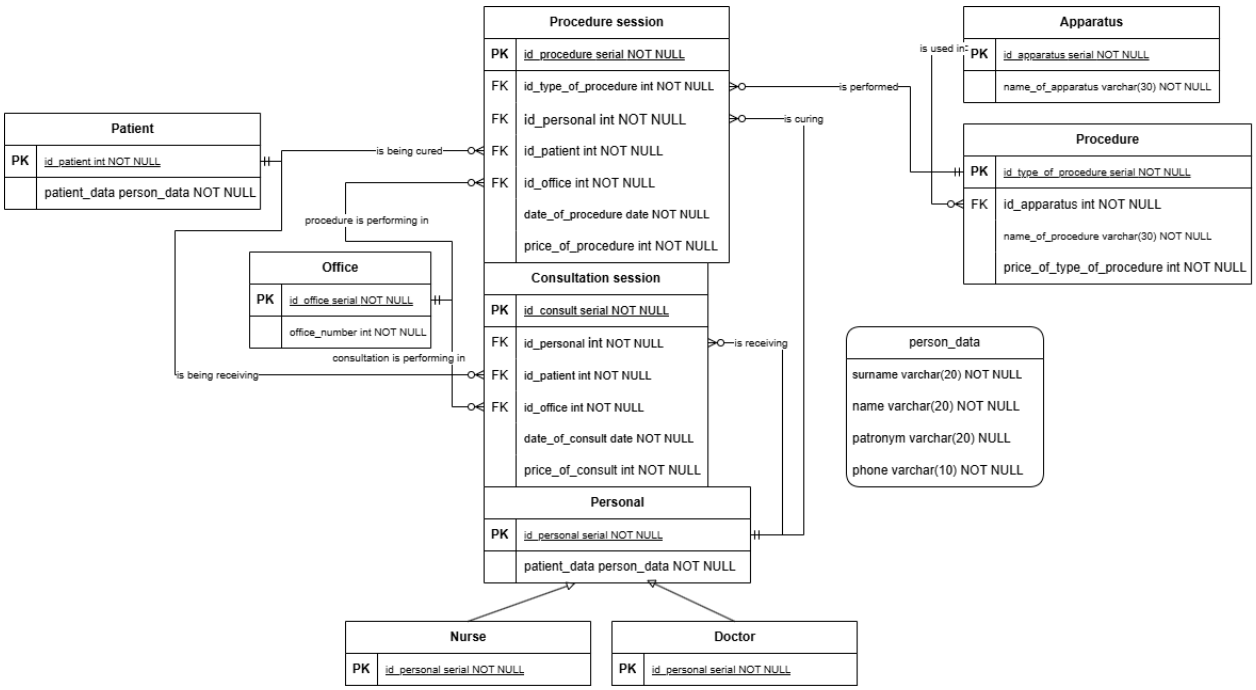


Рисунок 1 – Физическая модель

4. **Код создания базы**

```
CREATE TYPE PersonData AS (
   surname VARCHAR(20),
   name VARCHAR(20),
   patronym VARCHAR(20),
   phone VARCHAR(10)

);

CREATE TABLE Medical_Staff (
   id_personal SERIAL PRIMARY KEY,
   person PersonData NOT NULL
);

CREATE TABLE Doctor (
 PRIMARY KEY (id_personal)
) INHERITS (Medical_Staff);

CREATE TABLE Nurse (
 PRIMARY KEY (id_personal)
) INHERITS (Medical_Staff);

CREATE TABLE Patient (
   id_patient SERIAL PRIMARY KEY,
   person PersonData NOT NULL
);

CREATE TABLE Office (
   id_office SERIAL PRIMARY KEY,
   office_number INT NOT NULL
);

CREATE TABLE Apparatus (
```

```
    id_apparatus SERIAL PRIMARY KEY,
    name_of_apparatus VARCHAR(30) NOT NULL
);

CREATE TABLE Procedure_Type (
    id_type_of_procedure SERIAL PRIMARY KEY,
    name_of_procedure VARCHAR(30) NOT NULL,
    price_of_type_of_procedure INT NOT NULL,
    id_apparatus INT NOT NULL,
    CONSTRAINT fk_apparatus
        FOREIGN KEY (id_apparatus) REFERENCES Apparatus(id_apparatus)
        ON DELETE RESTRICT ON UPDATE RESTRICT
);

CREATE TABLE Consultation_Session (
    id_consult SERIAL PRIMARY KEY,
    id_personal INT NOT NULL,
    id_patient INT NOT NULL,
    id_office INT NOT NULL,
    price_of_consult INT NOT NULL,
    date_of_consult DATE NOT NULL,
    CONSTRAINT fk_patient_consult
        FOREIGN KEY (id_patient) REFERENCES Patient(id_patient)
        ON DELETE RESTRICT ON UPDATE RESTRICT,

    CONSTRAINT fk_office_consult
        FOREIGN KEY (id_office) REFERENCES Office(id_office)
        ON DELETE RESTRICT ON UPDATE RESTRICT
);

CREATE TABLE Procedure_Session (
    id_procedure SERIAL PRIMARY KEY,
    id_personal INT NOT NULL,
    id_patient INT NOT NULL,
    id_office INT NOT NULL,
```

```
    id_type_of_procedure INT NOT NULL,
    price_of_procedure INT NOT NULL,
    date_of_procedure DATE NOT NULL,

    CONSTRAINT fk_patient_proc
        FOREIGN KEY (id_patient) REFERENCES Patient(id_patient)
        ON DELETE RESTRICT ON UPDATE RESTRICT,

    CONSTRAINT fk_office_proc
        FOREIGN KEY (id_office) REFERENCES Office(id_office)
        ON DELETE RESTRICT ON UPDATE RESTRICT,

    CONSTRAINT fk_type_proc
        FOREIGN KEY (id_type_of_procedure) REFERENCES
Procedure_Type(id_type_of_procedure)
        ON DELETE RESTRICT ON UPDATE RESTRICT
);

CREATE OR REPLACE FUNCTION check_med_staff()
RETURNS TRIGGER AS $$
BEGIN
 IF NOT EXISTS (SELECT 1 FROM medical_staff WHERE id_personal =
NEW.id_personal) THEN
            RAISE EXCEPTION 'Указанного сотрудника не существует';
 END IF;

 RETURN NEW;
END
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER check_staff_consult
BEFORE INSERT OR UPDATE ON consultation_session
FOR EACH ROW EXECUTE FUNCTION check_med_staff();

CREATE OR REPLACE TRIGGER check_staff_procedure
```

```
BEFORE INSERT OR UPDATE ON procedure_session
FOR EACH ROW EXECUTE FUNCTION check_med_staff();

CREATE OR REPLACE FUNCTION check_staff_delete()
RETURNS TRIGGER AS $$
BEGIN
 IF EXISTS (SELECT 1 FROM consultation_session WHERE id_personal =
OLD.id_personal) THEN
        RAISE EXCEPTION 'На сотрудника записаны приёмы';
 END IF;

 IF EXISTS (SELECT 1 FROM procedure_session WHERE id_personal =
OLD.id_personal) THEN
        RAISE EXCEPTION 'На сотрудника записаны процедуры';
 END IF;

 RETURN OLD;
END
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER check_staff_consult_delete
BEFORE DELETE ON doctor
FOR EACH ROW EXECUTE FUNCTION check_staff_delete();

CREATE OR REPLACE TRIGGER check_staff_procedure_delete
BEFORE DELETE ON nurse
FOR EACH ROW EXECUTE FUNCTION check_staff_delete();

CREATE OR REPLACE FUNCTION check_staff_update()
RETURNS TRIGGER AS $$
BEGIN
 IF (NEW.id_personal IS DISTINCT FROM OLD.id_personal) THEN

        IF EXISTS (SELECT 1 FROM consultation_session WHERE id_personal =
OLD.id_personal) THEN
```

```
            RAISE EXCEPTION 'На сотрудника записаны приёмы';
        END IF;

        IF EXISTS (SELECT 1 FROM procedure_session WHERE id_personal =
OLD.id_personal) THEN
            RAISE EXCEPTION 'На сотрудника записаны процедуры';
        END IF;

 END IF;

 RETURN NEW;
END
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER check_staff_consult_update
BEFORE UPDATE ON doctor
FOR EACH ROW EXECUTE FUNCTION check_staff_update();

CREATE OR REPLACE TRIGGER check_staff_procedure_update
BEFORE UPDATE ON nurse
FOR EACH ROW EXECUTE FUNCTION check_staff_update();
```

5. **Выводы**

   В процессе лабораторной работы были получены практические навыки по проектированию и созданию баз данных в СУБД PostgreSQL.